◆ **AEM Content Sync**

DEPLOYMENT & OPERATIONS RUNBOOK

Deployment & Publishing Runbook

Clone, build, deploy, wire the permissions, run & test, promote to Production, and publish the App Builder app to your organization's catalog — for AEM as a Cloud Service. Written so someone new to the project can follow it end to end.

PROJECT
AEM Content Sync

REPOSITORY
github.com/rwunsch/aem-content-sync

APP TYPE
App Builder · dx/excshell/1

LICENSE
Apache-2.0

Contents

1 · What this is & what App Builder is

The Content Sync app wraps Cloud Manager's **Content Copy** in a scheduled, observable, role-gated pipeline that keeps a lower environment (Stage) aligned with Production — including the *published* state, which Content Copy alone does not carry across.

Adobe App Builder, in brief

App Builder is Adobe's serverless framework for building custom apps that extend Experience Cloud. An app has two halves:

- **A front end** — a single-page app (here, React + React Spectrum) served from Adobe's CDN.
- **A back end** — serverless functions ("actions") running on **Adobe I/O Runtime** (Apache OpenWhisk). Actions can be web-facing or private, and can be chained, scheduled (alarms/cron), and gated by IMS auth.

This app is packaged as a `dx/excshell/1` **Experience Cloud Shell extension** (not a standalone application). That packaging is what lets the shell at `experience.adobe.com` load the SPA in an iframe and perform the IMS token / org handshake — which is the prerequisite for the gated back-end actions to be callable from the UI. It runs entirely on Adobe infrastructure; there are no customer-hosted servers.

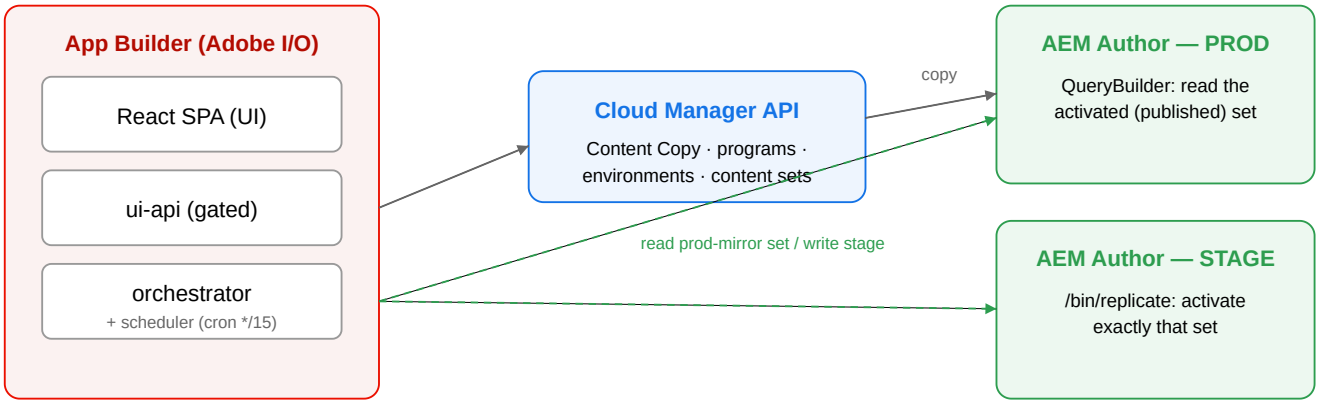

Figure 1 — The app holds both environments' credentials and talks to Cloud Manager (copy) and each AEM author (prod-mirror publish) directly.

Why "prod-mirror" publishing

Cloud Manager Content Copy is **author-tier only** and **strips replication status** (`cq:lastReplicated*`, `cq:lastReplicationAction*`, `cq:lastReplicatedBy*`) — by design, because that metadata describes the *source's* publish tier. So after a copy the destination author has no record of what was published, and nothing is published on Stage. The app's **prod-mirror** mode reads Production's activated set (QueryBuilder, per agent: publish/preview) and replicates exactly that set on Stage — reproducing production's live published state every run.

2 · Prerequisites & tooling

You need	Notes
Node.js 18+ and npm	The actions run on `nodejs:18`; build with a matching local Node.
git	To clone the repository.
Adobe I/O CLI (`aio`)	Use the **project-local** one (`./node_modules/.bin/aio`) — a global `aio` may resolve an incompatible Parcel and break the dev server / deploy.
Developer Console access	Rights to create an App Builder project, add APIs, and create credentials in your organization.
Admin Console access	To attach product profiles to the API credential (Cloud Manager roles + AEM Administrators per environment).
Cloud Manager program	A program with at least a source (Prod) and destination (Stage) **author** environment, and a **Content Set** defining the paths to copy.
AEM Service Credentials	Per environment, downloaded from each environment's Developer Console — used for the publish step (see §4 & §5).

> **Two "Developer Consoles"**
> There is the **Adobe Developer Console** (`developer.adobe.com/console`) where the App Builder project, its APIs, and the Cloud Manager *OAuth S2S* credential live; and there is each **AEM environment's Developer Console** (reached from Cloud Manager) where you download that environment's *AEM Service Credentials*. They are different things — §4 explains why you need both.

3 · Clone & explore the repository

1 Clone and install

```
git clone https://github.com/rwunsch/aem-content-sync.git
cd aem-content-sync
npm install            # also runs a postinstall that defuses an events-plugin deploy
crash
```

2 What's in the box

Path	What it is
`app.config.yaml`	Declares the app as a `dx/excshell/1` extension; includes `ext.config.yaml`.
`ext.config.yaml`	The runtime manifest — actions (`orchestrator` , `ui-api`), the scheduler trigger/rule, web assets, and the `post-app-deploy` hook.
`actions/`	Back-end: `orchestrator/` (the engine), `ui-api/` (the gated API), `utils/` (auth, cm-api, state, authz, config).
`web-src/src/`	React SPA — `App.js` , `components/` (Configure, Settings, Help), `exc-runtime.js` (the shell loader).
`config/content-sync.json`	Deploy-time default config (placeholders); the UI overrides it at runtime in I/O State.
`scripts/`	Build helpers + `ensure-auth-sequence.js` (the self-healing auth-wrapper hook).
`docs/`	This runbook, the troubleshooting guide, and architecture notes.

4 · The permission model (read this first)

Three independent authorization concerns, each granted in a different place. Getting one wrong fails in a different way — so understand them before you start.

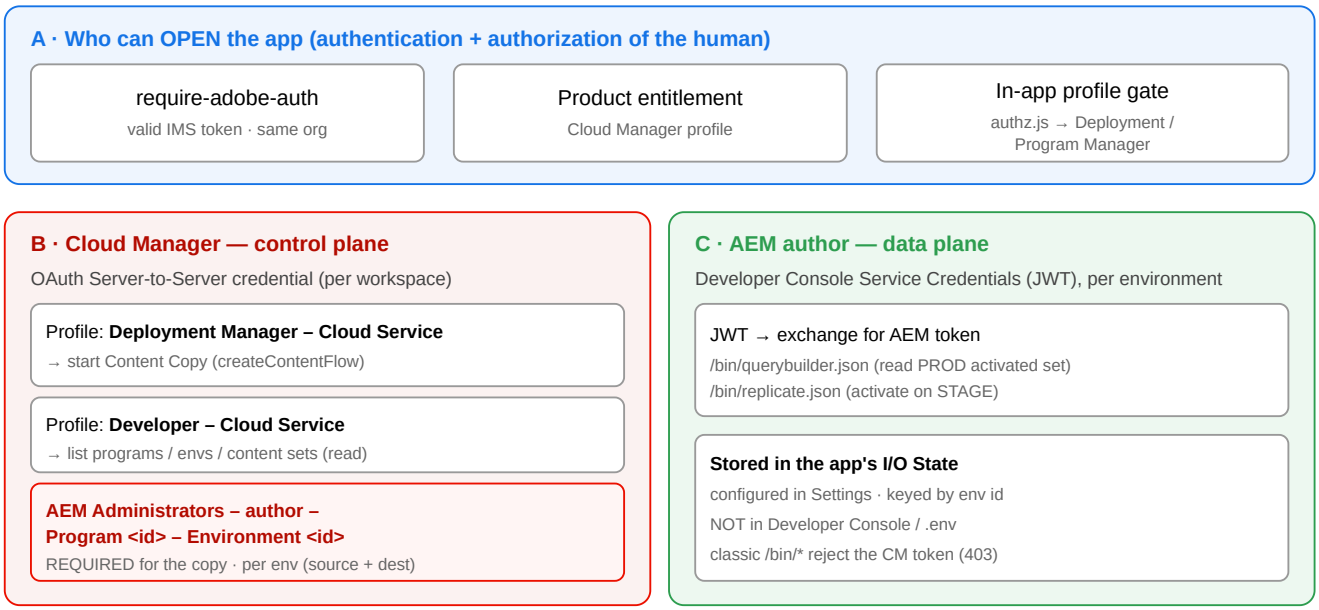

Figure 2 — How the permissions fit together. **B** drives Cloud Manager (the copy); **C** drives the AEM authors (the prod-mirror publish); **A** controls who may open the app at all.

> **The one that bites everyone**
>
> The Cloud Manager *Deployment Manager* profile is **necessary but not sufficient** for Content Copy. The API credential must **also** be a member of the `AEM Administrators – author – Program <id> – Environment <id>` product profile, for **both** the source and destination environments. Read calls succeed without it; only `createContentFlow` checks it — so the classic symptom is *"listing works but the copy returns 403."*

The two-token model

Token	How it's obtained	Used for
Cloud Manager token	`client_credentials` grant at `ims/token/v3`, using the S2S client id + secret	Cloud Manager API — copy, programs, environments, content sets
AEM author token	signed **JWT** exchanged at `ims/exchange/jwt`, using the per-environment Service Credentials	Classic AEM `/bin/replicate` & `/bin/querybuilder` (prod-mirror publish)

What a "Service Credential" is: an AEM-as-a-Cloud-Service environment exposes a per-environment *Service Credentials* JSON (technical account + private key + metascopes) from its Developer Console. The app signs a JWT with that private key and exchanges it for an AEM access token. This is separate from the Cloud Manager OAuth credential because the classic AEM `/bin/*` endpoints do not accept the Cloud Manager token.

5 · Developer Console — project, integration, API & credentials

1 Create the App Builder project

In the Adobe Developer Console, create a new project from the **App Builder** template (this provisions an I/O Runtime namespace). It comes with a **Stage** workspace; you'll add a **Production** workspace later (§11). Each workspace has its *own* namespace and credentials.

2 Add the APIs

To the workspace, add: **Cloud Manager** (drives Content Copy) and the **I/O Management API** (required for App Builder deploy/registration). Adding the Cloud Manager API is what creates the OAuth Server-to-Server credential. The app also uses **Adobe I/O State** (it stores jobs & credentials there) — see the callout for how that is provisioned.

> **State & Files are runtime services, not catalog cards**
> State (and **Files**, if you extend storage) bind to the project's **App Builder runtime namespace** — `@adobe/aio-lib-state` initialises from the namespace credentials, not a separate API key. The canonical provisioning adds the service *by code*: `aio console workspace api add --service-code CloudManagerSDK,AdobeIOManagementAPISDK,StateSDK`. If the Developer Console "Add API" *catalog* does not show a clickable **State**/**Files** card, that is expected and **not a blocker** — the libraries bind to the namespace once the app is deployed. Don't wait for a State card to appear.

3 Create the OAuth Server-to-Server credential & attach Cloud Manager profiles

When adding Cloud Manager, choose **OAuth Server-to-Server**. On its *Product Profiles* tab attach **Deployment Manager – Cloud Service** (needed to start a copy) and **Developer – Cloud Service** (read). This is credential **B** in Figure 2.

4 Add the credential to the AEM Administrators profile — per environment critical

In the **Admin Console** → **Products**, open the product profile named `AEM Administrators – author – Program <id> – Environment <id>`. On its **API credentials** tab choose **Add API credentials** and add the OAuth S2S credential from step 3. **Do this for both the source and the destination environment.** Without it, `createContentFlow` returns 403 even though listing works. Allow ~10–60 min to propagate.

> **If listing works but the copy 403s**
> This is almost always the per-environment AEM Administrators membership not (yet) in effect. Note the copy is initiated against the **source** environment, so the source env's profile is the one the call checks first.

5 Download each environment's AEM Service Credentials

From **Cloud Manager** → **the environment** → **Developer Console**, download the environment's **Service Credentials (JWT)** JSON. You'll paste these into the app's Settings (§8). These are credential **C** in Figure 2 — one per environment the job touches.

6 · Local development

1 Sign in & select the Stage workspace

```
aio login                # browser IMS login (the cli context)
aio app use              # pick your project → Stage workspace; writes .aio + .env
```

2 Map the Cloud Manager credential into action inputs

`aio app use` writes the IMS S2S *context* lines but **not** the convenience vars the actions read. Set them in `.env` from the context values (strip the brackets on the secret):

```
IMS_ORG_ID       = AIO_ims_contexts_<ctx>_ims__org__id
CM_CLIENT_ID     = AIO_ims_contexts_<ctx>_client__id
CM_CLIENT_SECRET = AIO_ims_contexts_<ctx>_client__secrets   # strip [ ]
```

Verify by minting a token: a `client_credentials` POST to `ims-na1.adobelogin.com/ims/token/v3` that returns a 200 with an `access_token` means the credential is good.

3 Run locally, inside the shell

```
aio app dev              # local dev server (https://localhost:9080)
```

Open the local build *through the shell* so it gets an IMS token:

```
https://experience.adobe.com/?devMode=true#/custom-apps/?
localDevUrl=https://localhost:9080
```

> **WSL / Windows cert trust**
>
> The shell iframe won't load `https://localhost` unless the dev cert is trusted by the OS (you can't click through a cert warning in an iframe). Regenerate the dev cert with a proper SAN (`DNS:localhost,IP:127.0.0.1`) and add it to the OS trust store. Never open the raw `adobeio-static.net` URL directly — only the shell supplies the token.

7 · Deploy to Stage

```
aio app deploy          # build SPA + actions, deploy to the namespace, publish the extension
point
```

What you get, and the security model:

- **SPA** deployed to the CDN; **actions** deployed to the workspace's I/O Runtime namespace.
- `ui-api` is a **web action behind** `require-adobe-auth` — anonymous calls get **401**; only a valid IMS token from the same org is accepted.
- `orchestrator` is a **non-web (private)** action — no public URL; invoked only inside the namespace (the scheduler rule, the `ui-api` trigger op, and its own self-chain), all via the authenticated namespace API.
- The static bundle contains **no secrets**; all credentials live in action inputs / I/O State, server-side.

> **Stale CLI login**
> If `aio app deploy` fails with `IMSOAuthSDK:TIMEOUT` / `CANNOT_GENERATE_TOKEN`, the cached token expired — re-run `aio login`. (Action *code* can also be updated with the static Runtime namespace key via `aio rt` / the OpenWhisk REST API, but web/CDN deploy and catalog publish need a real login.)

8 · Configure jobs & credentials in the app

Open the deployed app in the shell, then:

1 Settings → AEM Service Credentials

Paste each environment's Service Credentials JSON (from §5 step 5) into its box and Save. Use **Check credentials** to confirm. These are stored encrypted server-side and never returned to the browser.

> **The technical-account user doesn't exist in AEM until "Check credentials" succeeds**
> JWT SSO provisions the technical-account user on its **first authenticated login** — so before any successful call it is *not* in the environment's user list, and you cannot find it to grant rights. The order is: paste the JSON → **Save** → **Check credentials** (this authenticates and provisions the user) → *then* grant it replication rights (add it to a group with `crx:replicate` on the publish paths, least privilege, or to the env's `AEM Administrators` profile). In the **Admin Console** you add the **API credential** to the `AEM Administrators – author – Environment <id>` profile — you do **not** search for a human user by email.

2 Configure → the job

Pick the **program**, the **source** → **destination** environments, the **content set(s)** (author-tier; each carries a publish flag and an optional *wipe destination*), the **publish mode** (§9), and the **cron schedule**. Changes autosave to I/O State — no redeploy needed.

> **Content sets: the app keys by id, the console shows the name**
> Cloud Manager's UI labels content sets by **name**, but the app and the API key them by **id**. When Cloud Manager is reachable, the Configure picker resolves this for you — it lists each set as *name (id)* and fills its root paths read-only. If the picker is empty it falls back to a manual **Content set ID** field, which means Cloud Manager isn't reachable yet (fix that first — §13). To read the ids directly: `GET https://cm.adobe.io/api/program/<programId>/contentSets` (the JSON carries both `id` and `name`).

3 Settings → Access control

Choose which **Admin Console profiles** may use the app (default: Cloud Manager **Deployment & Program Managers**). You can add others, enter a custom IMS group, or open it to any signed-in org user. The server refuses a selection that would lock *you* out.

9 · Publish modes

After the copy, the publish step reproduces a published state on the destination. Choose the mode per job.

Mode	What it does	Use when
prod-mirror (default)	Queries the *source* (Prod) for its activated set per agent (publish / preview) and replicates *exactly that set* on the destination — tracking new publishes and removals each run, excluding drafts and preview-only content.	You want Stage to faithfully mirror what is live on Prod.
tree activation	Server-side tree activation from a root path. With *onlyActivated* it activates only nodes already marked activated; otherwise it walks and activates the tree.	Large subtrees where a server-side walk is more efficient than path-by-path.
publish everything	Activates every path in the content set, regardless of source publish state.	The content set is curated so "everything in it should be published."

Targets & options

Each job can target **publish** and/or **preview**, include children, set chunk sizes, and run as a **dry run** (reports what it would do without replicating). References are *not* followed automatically — include referenced paths in the content set if you need them.

Scale note

prod-mirror enumerates Prod over QueryBuilder (HTTP) and replicates in chunks within the 60-second action limit. This is fine for curated/targeted sets but does not scale cleanly to millions of nodes — at that scale the write side is bounded by the destination's replication-agent throughput regardless of tooling.

10 · Test thoroughly

1 Validate eligibility (read-only)

In **Help** → **Diagnostics** → **Validate eligibility**, pick a job. It counts how many nodes are currently activated for publish and for preview under each publish path — changing nothing. Use it to confirm the source holds the publish status a run will rely on, and to spot preview gaps.

2 Run once, manually

Trigger the job from the UI (*Run now*). The scheduler also fires on its cron, but a manual run is the fastest test. Watch the run progress through the state machine:

Figure 3 — Run state machine. COPYING calls Cloud Manager; PUBLISHING drives the prod-mirror replication; failures route to NOTIFYING_FAILURE → IDLE.

Progress, per-job status and the last ~50 log lines are visible in the UI. Cloud-Manager content-flow status is `IN_PROGRESS / COMPLETED / FAILED / CANCELLED`.

3 Confirm the outcome

- The content-flow reaches **COMPLETED** and the run reaches **NOTIFYING_SUCCESS** → **IDLE**.
- On the destination author, the copied paths exist and (for the activated subset) show as activated for the chosen agents.
- Re-run **Validate eligibility** on the destination and confirm the counts match the source's published set.
- Try a **dry run** first on a new job to preview scope without writing.

11 · Promote to Production

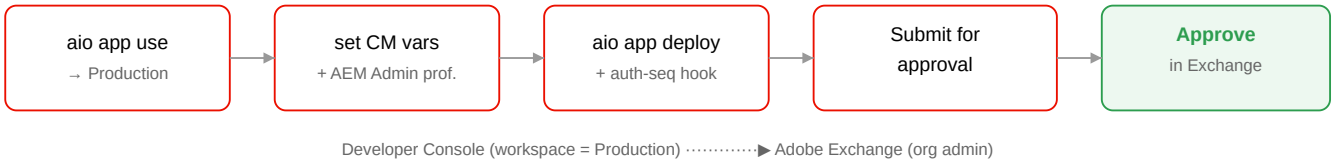

Figure 4 — Production promotion and catalog publish flow.

1 Set up the Production workspace

In the Developer Console, the Production workspace needs its **own** OAuth S2S credential and the same APIs as Stage (Cloud Manager, I/O Management, State), plus — repeating §5 step 4 — the **Deployment Manager** profile *and* the per-environment **AEM Administrators** profiles. It has a separate runtime namespace.

2 Switch the project to Production

```
cp .env .env.stage.bak ; cp .aio .aio.stage.bak    # back up Stage first
aio app use -w Production --overwrite --no-input    # rewrites .env/.aio to Production
```

> **It overwrites local config**
> Your deployed Stage app keeps running; only the local `.env` / `.aio` flip. Re-map the CM convenience vars (§6 step 2) for the Production credential. Return to Stage later with `aio app use -w Stage` or by restoring the backups.

3 Deploy to Production

```
aio app deploy
```

> **Fresh-namespace gotcha (auto-healed)**
> On a brand-new namespace the deploy can create the inner `__secured_ui-api` action but skip the `require-adobe-auth` `ui-api` *wrapper sequence* → the SPA's calls 404 → UI shows "backend unreachable." The bundled `post-app-deploy` hook (`scripts/ensure-auth-sequence.js`) re-creates it automatically after every deploy.

4 Seed the Production configuration & test

The Production namespace starts empty — repeat §8 (Service Credentials, job, access control) and §10 (test) there. It has its own I/O State, independent of Stage.

> **Updating an already-published app**

> After the app is published (§12), `aio app deploy` will **skip** deploying to Production to protect the live app. Push later code/config updates with `aio app deploy --force-deploy` — the catalog listing stays approved; only the namespace code updates. (No re-approval needed.)

12 · Publish to the organization catalog

A `devMode` / `localDevUrl` link is a one-shot dev side-load. For a durable, reloadable entry point in the Experience Cloud catalog, publish the app — a submission in the Developer Console followed by an org-admin approval in Adobe Exchange.

1 Submit for approval (Developer Console)

Developer Console → **Production** workspace → left nav **Approval**. Complete the *App Submission Details* form: **App title** (becomes the custom URL), **description**, **contact email**, a **512×512 PNG/JPG icon ≤100 kb** (required), and a **note to reviewer** (also required). Click **Submit** — status becomes *In Review*.

2 Approve as an org administrator (Adobe Exchange)

Go to exchange.adobe.com/manage → **App Builder applications**. Switch to the **Private** distribution tab (first-party in-org apps live there, not Public). Find your app showing *Pending Review*, click **Review**, then **Approve** (the dialog also offers Reject / Cancel). The status flips to `Approved` and the row's action becomes *Revoke*.

> **Who can approve**
>
> The submitter and the approver can be the same person if you hold the org **System Administrator** role; otherwise an org admin performs the Exchange approval.

3 Result — stable catalog URL

The app appears under the org's **App Builder Apps** with a durable, reloadable URL (no more `devMode`):

```
https://experience.adobe.com/#/@<org>/custom-apps/<namespace>/
```

End users still need the product entitlement *and* one of the configured Admin Console profiles (Figure 2-A). To restrict who sees it, manage the Cloud Manager product profile membership in the Admin Console.

13 · Operations & troubleshooting

> **After any permission change → redeploy**
> The action caches the Cloud Manager token (~24 h) and IMS bakes entitlements in at *mint time*. A warm container keeps using a token minted *before* your permission change. After granting/removing a role or profile, wait for propagation, then run `aio app deploy` — the next invocation runs in a fresh container and mints a token with the updated entitlements.

Symptom	Cause & fix
"Can't reach Cloud Manager" in Settings; no environments list; Configure content-set dropdown empty	The OAuth S2S credential can't list the program. Two usual causes: (a) the credential lacks the **Developer – Cloud Service** (read) and/or **Deployment Manager** profiles (§5 step 3); (b) **the deployed action doesn't have the credential values** — `aio app use` writes the IMS context but NOT `IMS_ORG_ID` / `CM_CLIENT_ID` / `CM_CLIENT_SECRET`; set them in `.env` (§6 step 2, strip the `[ ]` on the secret) and `aio app deploy`. Verify with a `client_credentials` token POST → 200.
Technical-account user **not found** in Admin Console / not present in the AEM instance	Expected before the first authenticated call — JWT SSO provisions the user on first login. Paste the env's Service Credentials → **Check credentials** (provisions it) → then grant replication rights. In Admin Console add the **API credential** to the env's `AEM Administrators` profile, not a human user (§8 step 1).
State / **Files** not offered in the "Add API" catalog	Not a blocker. They're App Builder runtime services bound to the namespace, not catalog APIs (§5 step 2); add by service code (`--service-code …,StateSDK`) if needed, or just proceed — the libraries bind on deploy.
Settings: **"Content-copy access … Not yet verified"** stays amber	By design until the *first successful run*; it auto-greens after one good copy. Don't chase it beforehand.
App **disappears on browser refresh** (opened via the `devMode` / `localDevUrl` link)	That link is a one-shot dev side-load — a refresh drops its query params. For a durable, reloadable URL, publish the app to the org catalog (§11–§12); it then lives at `experience.adobe.com/#/@<org>/custom-apps/<namespace>/`.
403 on the copy (`ERROR_CREATE_CONTENTFLOW`); listing works	The AEM Administrators per-env profile isn't in effect for the credential. Add it for the *source* and dest envs (§5 step 4), wait for propagation, `aio app deploy` (fresh token), retry.
"Backend unreachable" / the SPA's calls 404	The `require-adobe-auth` wrapper sequence wasn't created on a fresh namespace. The `post-app-deploy` hook fixes it; re-run `aio app deploy`, or recreate the `ui-api` sequence manually (see `docs/troubleshooting-shell-integration.md` §8).
401 on every call	Opened outside the shell (no IMS token). Open via the catalog or the `devMode` shell URL, not the raw `adobeio-static.net` URL.
403 "not authorized: profile"	The signed-in user lacks an allowed Admin Console profile. Adjust **Settings** → **Access control** or grant the user a profile.
"Failed to fetch" with a valid token	Historically a duplicated CORS header. The action must not set `Access-Control-*` — the platform owns CORS. (Already fixed in this codebase.)
`aio app deploy` auth timeout	Stale CLI login — `aio login` again.

Deploy silently "skipped" on Production; code doesn't update	Once the app is **published**, `aio app deploy` refuses to update the Production workspace ("deployment will be skipped … retract … to deploy updates"). Deploy updates with `aio app deploy --force-deploy` (no re-approval needed; the catalog listing is unaffected) — or retract in Adobe Exchange first.
"Concurrent content backflows are not allowed"	A previous flow is still active (or stuck). The app's `CHECK_STUCK` phase guards this; wait for the prior flow, or cancel it in Cloud Manager.

Full troubleshooting catalogue: `docs/troubleshooting-shell-integration.md` (shell integration, CORS, fresh-namespace auth sequence, CM credential vars, the Content Copy 403, and the token-refresh rule).